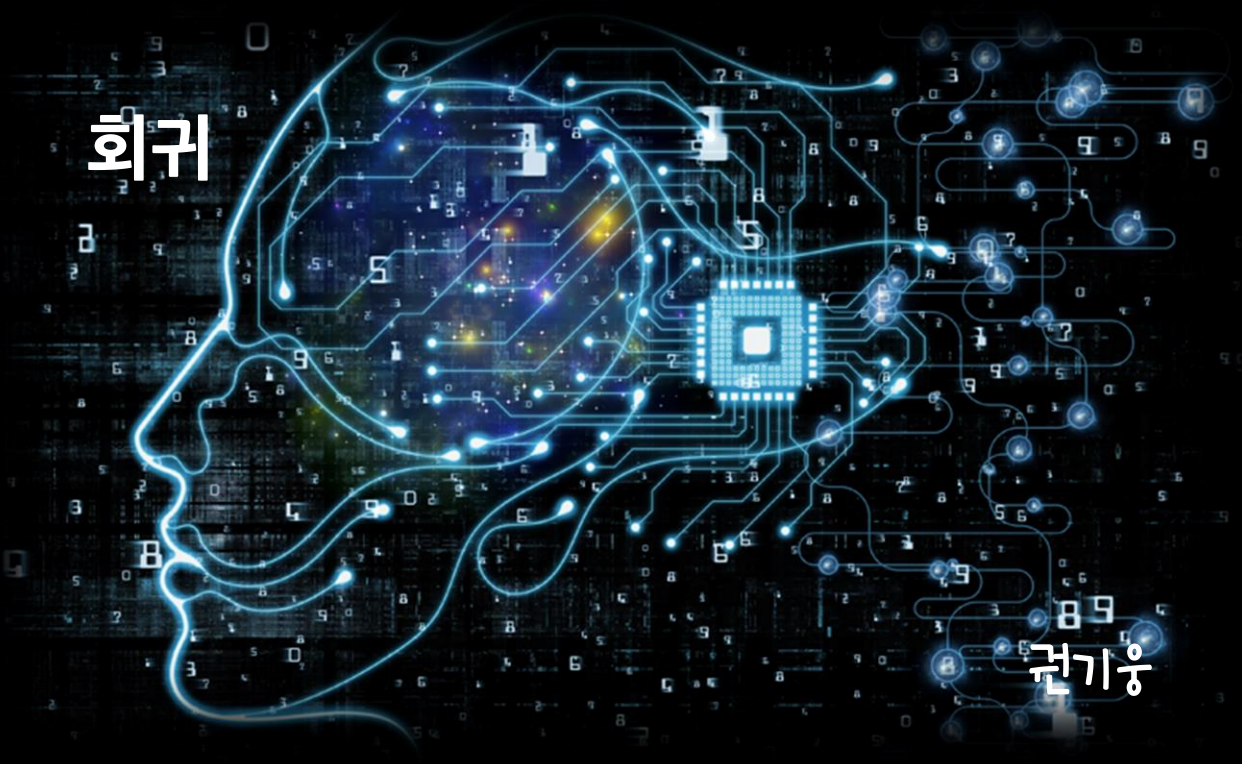
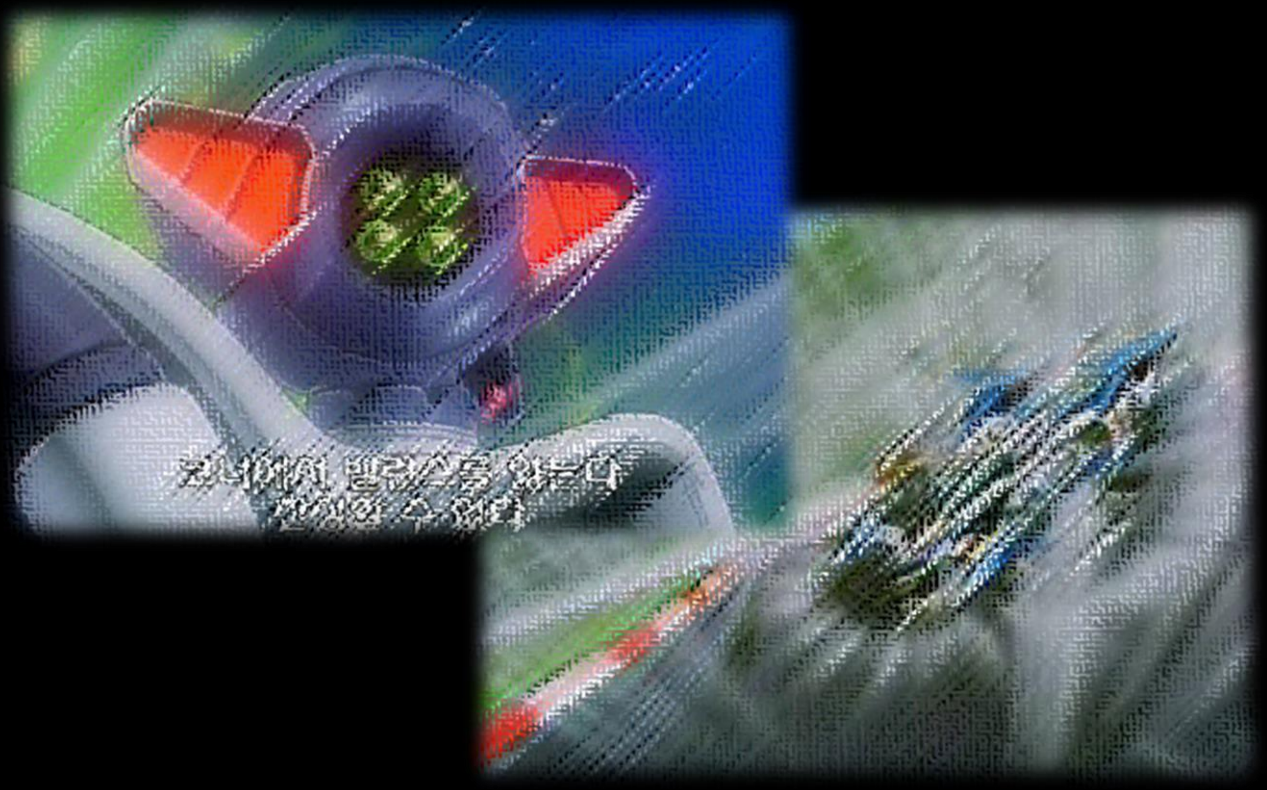
회귀



권기웅



TensorFlow ™

1. 회귀 분석

1.1 Regression

- 둘 또는 그 이상의 변수 사이의 관계 특히 변수 사이의 인과관계를 분석하는 주측통계의 한 분야
- 특정 변수값의 변화와 다른 변수값의 변화가 가지는 수학적 선형의 함수식을 파악함으로써 상호관계를 추론
- 추정된 함수식을 회귀방정식이라 함
- 이러한 회귀방정식을 통하여 회귀식이 타당시 되는 상황일 경우 독립변수의 값을 기초로 종속변수의 값을 예측할 수 있음

TensorFlow ™

2. 테스트 데이터로

2.1 테스트 데이터로

2.1.1 테스트 데이터로

```
# -*- coding:utf-8 -*-
import tensorflow as tf

xData = [1, 2, 3]
yData = [1, 2, 3]

# [1]짜리 list, -10~10의 균등분포 랜덤값
W = tf.Variable(tf.random_uniform([1], -10, 10))
b = tf.Variable(tf.random_uniform([1], -10, 10))

# 나중에 찾기 쉽게 name지정 가능
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)

# 식
sik = W * x + b
```

```
# 손실함수
# mean(h - Y) ^ 2 : 예측값(sik을 통해 계산한 값)과 실제값(y)의 거리를 비용(손실)함수로
distance = tf.reduce_mean(tf.square(sik - y))

# 텐서플로에 기본적으로 포함되어 있는 경사 하강법 최적화 메소드
optimizer = tf.train.GradientDescentOptimizer(0.1)

# 비용을 최적화 하는 것이 최종 목표
train_op = optimizer.minimize(distance)

sess = tf.Session()
sess.run(tf.global_variables_initializer())
# 최적화 100번 수행
for step in range(100):
    # 거리 구하고
    distanceVal = sess.run(distance, feed_dict={x:xData, y:yData})
    # 비용 최소화 시키기
    sess.run(train_op, feed_dict={x:xData, y:yData})
    ww = sess.run(W)
    bb = sess.run(b)
    print("%d) y = %fx + %f : 거리 %f" % (step, ww[0], bb[0], distanceVal))
```

```
# 테스트
print('테스트-----')
for i in range(5):
    xx = float(input("x : "))
    print("x : %f, y : %f" % (xx, sess.run(sik, feed_dict={x:xx})))
```

2.1.2 결과

```
90) y = 1.140689 * x + -0.319818, 거리:0.015405
91) y = 1.137307 * x + -0.312130, 거리:0.014673
92) y = 1.134006 * x + -0.304627, 거리:0.013976
93) y = 1.130784 * x + -0.297304, 거리:0.013312
94) y = 1.127640 * x + -0.290157, 거리:0.012680
95) y = 1.124572 * x + -0.283182, 거리:0.012078
96) y = 1.121577 * x + -0.276374, 거리:0.011504
97) y = 1.118655 * x + -0.269730, 거리:0.010958
98) y = 1.115802 * x + -0.263246, 거리:0.010437
99) y = 1.113019 * x + -0.256918, 거리:0.009941
테스트-----
x : 5
x: 5.000000, y: 5.308175
```

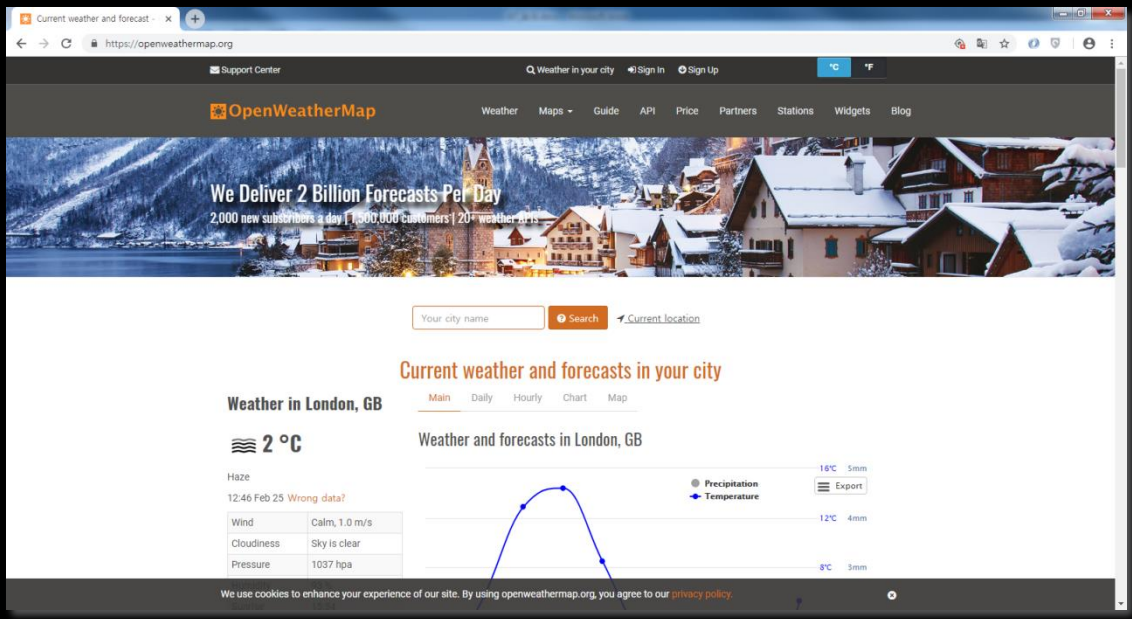
TensorFlow ™

3. 기온 예측

3.1 데이터 구하기

3.1.1 데이터 구하기

https://openweathermap.org/



3.1.2 JSON 파싱해서 파일로

```
# -*- coding:utf-8 -*-
from http.client import HTTPConnection
from json import loads

f = open('D:||Kwon||weather.csv', 'a', encoding='utf-8')
u, json = None, None
lat = 34.430272
hc = HTTPConnection('api.openweathermap.org')
while True:
    lat += 0.01
    if lat >= 37.811575:
        break
    u = '/data/2.5/weather?lat=%f&lon=126.9811354&appid=baff8f3c6cbc28a4024e336599de28c4&un
    hc.request('GET', u)
    json = hc.getresponse().read()
    weatherData = loads(json)
    data = str(weatherData['coord']['lat']) + ',' + str(weatherData['main']['temp']) + '\n'
    f.write(data)
    print(data)
f.close()
hc.close()
```

3.2 회귀

3.2.1 파일에서 읽어오기

```
f = open('D:||Kwon||weather.csv', 'r', encoding='utf-8')
weatherData = f.readlines()
lat = []
temp = []
for line in weatherData:
    line = line.split(',')
    lat.append(float(line[0]))
    temp.append(float(line[1].replace('\n', '')))

f.close()
```

3.2.2 회귀 분석

```
import tensorflow as tf

W = tf.Variable(tf.random_uniform([1], -10.0, 10.0))
b = tf.Variable(tf.random_uniform([1], -10.0, 10.0))
x = tf.placeholder(tf.float32, name='a')
y = tf.placeholder(tf.float32, name='b')

sik = W * x + b

distance = tf.reduce_mean(tf.square(sik - y))
optimizer = tf.train.GradientDescentOptimizer(0.0001)
train = optimizer.minimize(distance)

sess = tf.Session()
sess.run(tf.global_variables_initializer())

for step in range(5000):
    d = sess.run(distance, feed_dict={x:lat, y:temp})
    sess.run(train, feed_dict={x:lat, y:temp})
    print('%d) %fx + %f : 거리 %f' % (step, sess.run(W), sess.run(b), d))
```

3.2.3 결과 구하기

```
lat2 = float(input('위도 : '))
temp2 = sess.run(sik, feed_dict={x:lat2})
print('위도 %f는 %f℃' % (lat2, temp2))
```

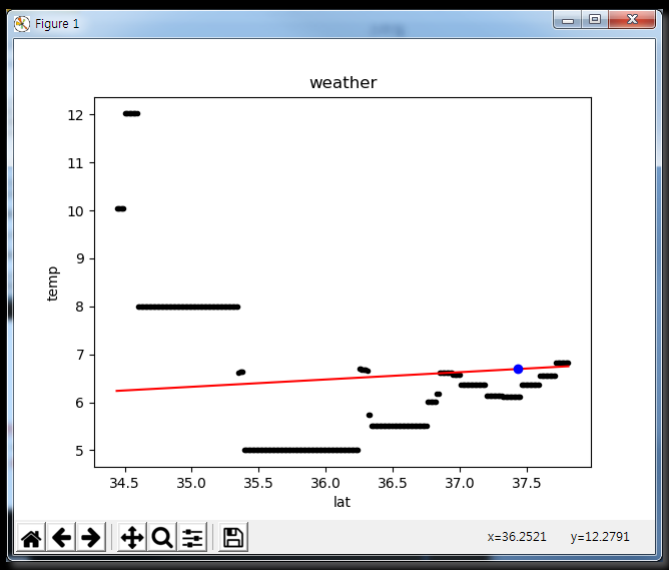
```
Console
<terminated> regression.py [C:₩Users₩soldesk₩AppData₩Local₩Programs₩Python₩Python36₩python.exe]
4993) 0.182808x + -0.110829 : 거리 2.638670
4994) 0.182808x + -0.110824 : 거리 2.638670
4995) 0.182808x + -0.110819 : 거리 2.638669
4996) 0.182808x + -0.110814 : 거리 2.638669
4997) 0.182808x + -0.110809 : 거리 2.638669
4998) 0.182808x + -0.110804 : 거리 2.638669
4999) 0.182807x + -0.110799 : 거리 2.638669
위도 : 37.54521
위도 37.545210는 6.752744℃
```

3.2.4 그래프 그리기

일단 그래프 라이브러리 설치하고

명령프롬프트에서 pip3 install matplotlib

```
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:₩Users₩soldesk>pip3 install matplotlib
Collecting matplotlib
  Downloading https://files.pythonhosted.org/packages/b1/56/569c83515c10146fd0aa09e086816b12e301d081
7ba9a/matplotlib-3.0.2-cp36-cp36m-win_amd64.whl (8.9MB)
    100% |████████████████████████████████| 8.9MB 2.2MB/s
Requirement already satisfied: numpy>=1.10.0 in c:₩users₩soldesk₩appdata₩local₩programs₩python₩pytho
ckages (from matplotlib) (1.16.1)
Collecting cycler>=0.10 (from matplotlib)
  Downloading https://files.pythonhosted.org/packages/f7/d2/e07d3ebb2bd7af696440ce7e754c59dd546ffe1b
34e61/cycler-0.10.0-py2.py3-none-any.whl
Collecting pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.1 (from matplotlib)
  Downloading https://files.pythonhosted.org/packages/de/0a/001be530836743d8be6c2d85069f46fecf84ac6c
1d854/pyparsing-2.3.1-py2.py3-none-any.whl (61kB)
    100% |████████████████████████████████| 71kB 4.6MB/s
Collecting kiwisolver>=1.0.1 (from matplotlib)
  Downloading https://files.pythonhosted.org/packages/44/72/16630c3392eba03788ad87949390516bbc488e8e
d21a6/kiwisolver-1.0.1-cp36-none-win_amd64.whl (57kB)
    100% |████████████████████████████████| 61kB 3.9MB/s
Collecting python-dateutil>=2.1 (from matplotlib)
  Downloading https://files.pythonhosted.org/packages/41/17/c62faccbfbd163c7f57f3844689e3a78bae1f403
87fbb/python_dateutil-2.8.0-py2.py3-none-any.whl (226kB)
```

```python
import matplotlib.pyplot as plt

plt.title('weather')
plt.xlabel('lat')
plt.ylabel('temp')

plt.plot(lat, temp, 'k.') # 까만점
plt.plot(lat, sess.run(W) * lat + sess.run(b), 'r-') # 빨간선
plt.plot([lat2], [temp2], 'bo') # 파란동그라미

plt.show()
```